

Pi-CloudDOOM — Jury Demo Guide

Audience: the jury, tomorrow morning. **Goal:** show that the CI/CD pipeline is real, ends-to-ends, and resilient — without praying for production. **Format:** copy-paste commands, expected outputs, talking points.

0. Tabs to open *before* the jury arrives

Open these in browser tabs in this order so you can switch with **Ctrl+Tab**:

1. **GitHub repo** → <https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM>
 2. **GitHub Actions tab** → <https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM/actions>
 3. **GitHub Security tab** → <https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM/security/code-scanning>
 4. **GitHub Environments tab** → <https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM/deployments>
 5. **DockerHub** → <https://hub.docker.com/u/azizbna>
 6. **Production frontend** → <https://interviewprep-tn.me>
 7. **Production API health** → <https://api.interviewprep-tn.me/api/public/health>
 8. **Grafana** (if exposed) — your in-cluster URL
 9. A **local terminal** in the repo: `cd ~/Documents/4\ ArcTic13/S2/PICloud/PICloudDOOM/Pi-CloudDOOM`
 10. A **second terminal** with `kubectl` configured against the cluster (optional, advanced demo)
-

1. The 60-second story (your opener)

“We deploy a polyglot platform to **three different clouds** in one push: Spring services to our **OpenStack Kubernetes** cluster, AI services to **Azure Container Apps**, and the Angular frontend to **Vercel**. Everything is automated by **13 GitHub Actions workflows**: CI runs only on the services that changed, images are **signed with Cosign** and **scanned by Trivy**, and every deploy auto-rolls-back if smoke tests fail. Let me show you.”

2. Live demo — 4 scenes (12 min)

Scene 1 — “What happens on a PR” (3 min)

Goal: show that opening a PR runs CI + security scans + posts a Vercel preview URL.

```

# In your local terminal
cd "$(git rev-parse --show-toplevel)"
git checkout deployment/hybrid-openstack-azure-vercel
git pull
git checkout -b demo/jury-pr-$(date +%H%M)

# Touch the frontend so we get a Vercel preview AND CI on the frontend
echo "<!-- jury demo $(date) -->" >> frontend/src/index.html

# Touch a backend service so we get CI for it too
echo "// jury demo $(date)" >> user-service/src/main/java/com/userservice/UserServiceApplica

git add -A
git commit -m "demo: jury PR - frontend + user-service edits"
git push -u origin HEAD

```

Open the GitHub repo → **Pull requests** → “New pull request” from this branch → create.

What to point at: - Checks tab in the PR — show the running jobs: - ci / changes (paths-filter) - ci / user (build-user-service) - ci / frontend (build-frontend) - Vercel - Preview / Deploy Vercel Preview - CodeQL / analyze-* - Gitleaks / scan repo for secrets - **paths-filter intelligence**: open ci / changes → show outputs (user: true, frontend: true, others false). Say: “*We only ran CI for the two services that actually changed.*” - After ~2 min, the **bot comment** appears on the PR: > Vercel preview deployed — URL: <https://...vercel.app> - Click the URL → show the live preview frontend.

Talking points: - “Reusable workflows mean adding a new service is two lines.” - “If anything fails, branch protection blocks merge — **ci-summary** is the aggregate gate.” - “The bot updates one comment instead of spamming — [marocchino/sticky-pull-request-comment](#).”

Scene 2 — “What happens on merge to main” (4 min)

Goal: show the image build chain → 3-plane deploy.

You have two options here. Choose based on confidence:

Option A — Use a *past run* (safer; recommended for jury) In the Actions tab, open the most recent successful “**Build & Push images**” run.

What to point at: - The 10 parallel matrix jobs (build-user-service, ..., build-ollama-deploy) — each green. - Open one job log → show: - docker/setup-buildx-action step - docker/metadata-action output — the tag list (sha-<short>, <branch>, latest, semver). - docker/build-push-action

step → cache-from hits. - Trivy scan step → SARIF uploaded. - Sign image step → cosign sign --yes ... (no key needed!). - The **Step Summary** at the bottom: image, digest, tags. - Now click into the “**Deploy — OpenStack K8s**” run that auto-fired right after. - Show the **gate** job → resolved sha-<short>, namespace picloudboom. - Show the 7 parallel rollout jobs. - Open one: rollout-user-service → kubectl set image ..., kubectl rollout status ... --timeout=10m succeeded. - Open the smoke test job → user-service (Available, readyReplicas=1, port=8081) × 7.

Talking points: - “workflow_run event auto-links build → deploy. No human in the loop.” - “Smoke uses kubectl wait --for=condition=Available — that condition is exactly the readiness probe in the manifest, so we validate the same thing the cluster validates.” - “If smoke fails, we don’t roll back just one service — we roll back **all** 7, because they have inter-service dependencies.”

Option B — Trigger a live build (more impressive but slower) If you have 8 min of demo time and you’re confident:

```
GitHub → Actions → "Build & Push images" → Run workflow
  branch: deployment/hybrid-openstack-azure-vercel
  service: user-service          # single service = faster ( 4-5 min instead of 15)
```

Then wait. The deploy fires after build success.

Backup: if the build is slow, switch to Option A while waiting.

Scene 3 — “What happens when something breaks” (3 min)

Goal: prove the auto-rollback is real, not just YAML.

The safest, jury-friendly way: **show evidence from a past failure** (the commits b0a971be “bump rollout timeout 5m → 10m” and 83c29438 “bump picloudboom quota” were exactly this — quota was too tight, rollouts couldn’t create new pods, services rolled back automatically).

```
# In local terminal - show the deploy file's rollback logic
grep -n -A6 "Auto-rollback" .github/workflows/deploy-k8s.yml
```

Output:

```
- name: Auto-rollback on failure
  if: failure() && steps.setimg.outcome == 'success'
  run: |
    echo "::error::Rollout failed - rolling back ${ matrix.service } to previous rev
    kubectl -n "${ needs.gate.outputs.namespace }" rollout undo "deployment/${ matrix
    kubectl -n "${ needs.gate.outputs.namespace }" rollout status "deployment/${ matrix
```

```
# And the smoke-triggered full-namespace rollback:
grep -n -A11 "Roll back ALL services if smoke fails" .github/workflows/deploy-k8s.yml
```

Optional — Cause a rollback live (advanced) Only do this if you’ve practised it.

```
GitHub → Actions → "Deploy - OpenStack K8s" → Run workflow
  environment: prod (or dev to be safer!)
  sha: 0000000      # 7 zeros = tag that doesn't exist
  service: user-service
```

kubectl set image will pin a non-existent tag → pods can’t pull → rollout times out at 10 min → auto-rollback step runs → kubectl rollout undo restores the previous good revision.

WARNING: do this against `environment: dev` if you have a dev namespace populated. Against prod, you trigger a real 10-minute hang before rollback. Skip if you don’t have dev populated.

Talking points: - “Every deploy is reversible because every image is **immutable** — `sha-<short>` never moves. Rolling back is just pinning a previous SHA.” - “The pipeline auto-rolls back at three layers: single-service rollout, full-namespace on smoke fail, and traffic shift on ACA fail.”

Scene 4 — “Security, supply chain, observability” (2 min)

```
# Show signature verification - anyone with cosign installed can do this
docker pull docker.io/azizbna/pi-clouddoom-user-service:latest
```

```
cosign verify docker.io/azizbna/pi-clouddoom-user-service:latest \
  --certificate-identity-regex "https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM/.*" \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com \
  2>&1 | head -20
```

Expected: a JSON payload signed by the GitHub Actions OIDC issuer. If cosign isn’t installed: `brew install cosign`.

Switch to **GitHub Security** tab: - Show **Code scanning alerts** populated with: - CodeQL findings (java-kotlin, javascript-typescript, python, actions). - `trivy-k8s`, `trivy-infra` categories. - `kube-linter` category. - Show **Dependabot alerts** (weekly grouped PRs).

Talking points: - “Every image is signed *keylessly* — no private key to manage. The certificate is the OIDC identity of the GitHub Action that built it. Cryptographic proof of provenance.” - “Trivy scans for CVEs at image build time; we ship SARIF to GitHub. Day-1 we’re report-only while we triage; flipping to enforcing is a one-line YAML change.”

3. Backup demos (if you have extra time)

3.1 The pipeline's IaC

```
# Show the Heat template (OpenStack cluster as code)
ls -la infra/openstack/
head -100 infra/openstack/heat-cluster.yaml
```

“This file is the cluster. 5 nodes, Calico, Kubernetes 1.29.15, bootstraps itself via cloud-init + Ansible. Zero manual steps.”

3.2 The CI/CD README

Open docs/cicd/README.md in GitHub's rendered view. Show: - The ASCII pipeline diagram. - The 13-workflow table. - The 7-secret matrix. - The 9-step build history.

“Each commit c515dec9 ... 27de2227 corresponds to a doc explaining its design decisions and any OpenStack side-effects.”

3.3 The attack detector

```
# Show the original AI security component
ls k8s/attack-detection/
kubectl --kubeconfig ... -n attack-detection get pods,svc,serviceaccount
```

If Grafana is reachable: open the dashboard ai-attack-detector-k8s-dashboard.json — show the live attack_label metric.

4. Quick kubectl proofs (only if you have terminal access to the cluster)

```
# Set the kubeconfig (path will vary)
export KUBECONFIG=$HOME/.kube/config-picloudodoom
```

```
# 1. Show pods are running
kubectl -n picloudodoom get pods
```

```
# 2. Show every Spring service is pinned to a sha-<short> image (NOT :latest)
kubectl -n picloudodoom get deploy -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.spec
```

```
# 3. Show rollout history
kubectl -n picloudodoom rollout history deployment/user-service
```

```
# 4. Show the cicd-deployer RBAC scope (it's NAMESPACE-scoped - not cluster admin)
```

```
kubectl -n piclouddoom describe role cicd-deployer
```

```
# 5. Show the resource quota
```

```
kubectl -n piclouddoom describe quota piclouddoom-quota
```

5. Local pipeline simulation (no cluster needed)

If the cluster is down or you want to demo the *workflow logic* on your laptop:

5.1 Build an image locally exactly like CI does

```
cd "$(git rev-parse --show-toplevel)"
SVC=user-service
SHA=$(git rev-parse --short HEAD)

# Mimic the docker/build-push-action call
docker buildx build \
  --tag "azizbna/pi-clouddoom-${SVC}:sha-${SHA}" \
  --tag "azizbna/pi-clouddoom-${SVC}:deployment-hybrid-openstack-azure-vercel" \
  --provenance=mode=max \
  --sbom=true \
  --cache-from "type=local,src=/tmp/.buildx-cache/${SVC}" \
  --cache-to "type=local,dest=/tmp/.buildx-cache/${SVC},mode=max" \
  --load \
  ./${SVC}

docker images | grep ${SVC}
```

5.2 Trivy scan that image locally

```
# Install once: brew install trivy
```

```
trivy image --severity CRITICAL,HIGH --ignore-unfixed "azizbna/pi-clouddoom-${SVC}:sha-${SHA}"
```

Same scanner, same severity filter as CI.

5.3 Run smoke.sh in HTTP mode

```
# Test all 7 services through the public ingress
```

```
BASE_URL=https://api.interviewprep-tn.me bash scripts/smoke.sh
```

Expected output:

```
Pi-CloudDOOM smoke test
Mode: HTTP (base=https://api.interviewprep-tn.me)

user-service → https://api.interviewprep-tn.me/actuator/health (200)
interview-service → https://api.interviewprep-tn.me/actuator/health (200)
```

```
...
All 7 services healthy.
```

(Note: the prod ingress doesn't expose `/actuator/health` directly on every path; you may want to override `BASE_URL` per-service or test individual endpoints like `/api/public/health` for user-service. Adjust per your routing rules.)

5.4 Run a workflow file's logic line-by-line in your shell

For the rollback step, for example:

```
SVC=user-service
NS=picloudboom

# What the workflow does on failure:
kubectl -n "$NS" rollout undo deployment/"$SVC"
kubectl -n "$NS" rollout status deployment/"$SVC" --timeout=3m
```

5.5 Verify any image's signature in public

```
# Anyone, anywhere, can do this - no auth needed
cosign verify docker.io/azizbna/pi-cloudboom-interview-service:latest \
  --certificate-identity-regexp "https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM/.*" \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com
```

6. Q&A — likely jury questions & answers

Question	Answer
Why three clouds, not one?	Workloads have different requirements: heavy stateful infra fits OpenStack (free in-house), bursty AI fits ACA (scale-to-zero, pay-per-use), static frontend fits Vercel (CDN edge). Each plane plays to a strength.
What if the OpenStack cluster goes down?	The frontend stays up on Vercel; AI services stay up on Azure. Auth (Keycloak) and 7 Spring services go offline. This is documented in the day-1 limitations — HA is the post-launch priority #1.

Question	Answer
What stops a malicious commit from deploying?	(1) Branch protection — <code>ci-summary</code> , CodeQL, Gitleaks must pass. (2) CodeQL extended queries catch obvious patterns. (3) Gitleaks scans full history. (4) Trivy scans the resulting image. (5) Production GitHub Environment can require a reviewer. (6) Images are signed — anyone can verify provenance.
What's the rollback time?	Single service: ~30 s for <code>kubectl rollout undo</code> to schedule the old ReplicaSet + 1-3 min for Spring startup. ACA: ~10 s for <code>az ingress traffic set</code> . Vercel: instant — previous deployment never stops serving until new one is live.
Why not Argo CD / Flux?	GitOps is on the post-launch roadmap. Day-1 we wanted imperative deploys we could read line-by-line — easier to debug and to demo to a jury today.
How do you handle DB schema changes?	Flyway runs at Spring boot startup. Incompatible migrations crash-loop → auto-rollback kicks in. Pre-deploy DB migration tooling is on the backlog.
What's the cost model?	OpenStack: free (school infra). Azure ACA: pay-per-second of active CPU + provisioned memory; scale-to-zero so idle = \$0. Vercel: free hobby tier. DockerHub: free public repos. Total marginal cost ACA usage.
How do you rotate secrets?	Rotate the value in the GitHub Secrets UI; next workflow run picks it up. For <code>AZURE_CREDENTIALS</code> , an annual rotation is documented (or migrate to federated OIDC — already planned). Kubeconfig: re-run <code>extract-kubeconfig.sh</code> , update the secret.

Question	Answer
Why immutable sha-<i><short></i> tags?	Because mutable tags like <code>:latest</code> are unsafe — two pods of the same Deployment can run different code. With <code>sha-<i><short></i></code> every image is content-addressable and a rollback is just pinning a different tag.
What's signed? What's not?	All 10 backend/AI images we build are signed at push time. The frontend isn't an image (static on Vercel). Base images (postgres, redis, kafka, keycloak) are not signed by us; their provenance is upstream's responsibility.

7. Pre-demo checklist (run *tonight*)

1. Make sure the prod cluster is healthy

```
curl -sf https://interviewprep-tn.me >/dev/null && echo " frontend up" || echo " frontend I
curl -sf https://auth.interviewprep-tn.me/realms/myapp-realm >/dev/null && echo " keycloak
curl -sf https://api.interviewprep-tn.me/api/public/health >/dev/null && echo " api up" ||
```

2. Make sure recent workflow runs are green

```
gh run list --limit 10 --json status,conclusion,name,createdAt | head -50
# or open the Actions tab in browser
```

3. Make sure DockerHub has the latest images

```
docker pull docker.io/azizbna/pi-cluddoom-user-service:latest >/dev/null && echo " image p
```

4. Make sure cosign is installed locally

```
command -v cosign >/dev/null && cosign version | head -3 || echo "Install: brew install cosi
```

5. Make sure you can run the smoke script

```
ls -la scripts/smoke.sh && head -5 scripts/smoke.sh
```

6. Make sure your gh CLI is authed

```
gh auth status
```

7. Decide which past workflow run you'll demo in Scene 2 - note its URL

```
echo "Demo run URL: <paste here>"
```

8. Recovery playbook — if anything breaks live

Problem	Recovery
Prod URL <code>interviewprep-tn.me</code> doesn't respond	Switch the demo to localhost — <code>cd infra && docker compose up -d</code> — and demo the same flow there. The workflow files are the same.
<code>Build & Push images</code> is currently failing	Skip Scene 2's Option B. Use Option A (past successful run).
Vercel preview URL hasn't appeared	Refresh the PR page; the sticky comment updates async. Show the running <code>Vercel - Preview</code> job in the meantime.
<code>kubectl</code> doesn't connect	Drop terminal demo. The pipeline GitHub UI is enough — that's where the jury looks anyway.
Cosign verify fails	The image you picked may be an old one signed under a different identity. Try <code>sha-<short></code> of a recent commit.
Internet drops	Show the workflow YAML files locally in VS Code. The architecture is the story; the live click-through is the proof.

9. The closing one-liner

“Three clouds. Thirteen workflows. Ten signed images. Auto-rollback at three layers. Zero `kubectl` in human hands.”

10. Reference — all the URLs you might need

Thing	URL
Repo	https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM
Actions	https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM/actions
Build & Push runs	https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM/actions/workflows/build-and-push.yml

Thing	URL
Deploy K8s runs	https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM/actions/workflows/deploy-k8s.yml
Deploy ACA runs	https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM/actions/workflows/deploy-aca.yml
Security alerts	https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM/security/code-scanning
Environments	https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM/deployments
DockerHub	https://hub.docker.com/u/azizbna
Frontend prod	https://interviewprep-tn.me
API prod	https://api.interviewprep-tn.me
Keycloak prod	https://auth.interviewprep-tn.me/realms/myapp-realm
CI/CD docs	<code>docs/cicd/README.md</code> in the repo

Good luck tomorrow. You've got this.